



DVD Assignment Report

4-bit Array Multiplier from RTL to GDSII with Open-Source Tools

Author: Vishnubhotla Nitin Sri Siva Kalyan
Roll: 2024112012
E-mail:
nitin.vishnubhotla@research.iiit.ac.in

Author: Alapati Sai Pavan Adit
Roll: 2024102064
E-mail:
saipavanadit.alapati@students.iiit.ac.in

GitHub Repo: https://github.com/NitinGamerX/dvd_assignment

Contents

1 Introduction and Setup **2**

2 Part A: RTL Design and Verification **4**
2.1 Task 1: Full Adder 4
2.2 Task 2: Array Multiplier 4
2.3 Task 3: Self-Checking Testbench 8
2.4 Task 4: Error-Detection Test 9
2.5 Task 5: Worked Example and Waveform 10

3 Part B: Logic Synthesis with Yosys **11**
3.1 Task 1: Synthesis Script 11
3.2 Task 2: Synthesis Results 11
3.3 Task 3: Gate-Level Simulation 12

4 Part C: Static Timing Analysis with OpenSTA **13**
4.1 Task 1: STA Script and Maximum Frequency 13
4.2 Task 2: Critical Path 14
4.3 Task 3: Comparison with the $2N$ Full-Adder Estimate 14
4.4 Part C Results 15

5 Part D: Physical Design from RTL to GDSII **16**
5.1 Task 1: Configuration and Runs 16
5.2 Task 2: Physical Design Results 17
5.3 Task 3: Post-Route Maximum Frequency 17
5.4 Task 4: Cells Added by Place and Route 18
5.5 Task 5: Final Layout 19

1 Introduction and Setup

In this assignment we designed a 4-bit × 4-bit unsigned array multiplier (with an 8-bit product) in Verilog and took it all the way from RTL to a GDSII layout on the SkyWater SKY130 (130 nm) process. We used the `sky130_fd_sc_hd` standard cell library and only open-source tools for the whole flow.

The work is split into four parts, which follow the usual steps of a digital design flow:

- **Part A:** RTL design and exhaustive functional verification using `Icarus Verilog` and `GTKWave`.
- **Part B:** Logic synthesis to SKY130 standard cells using `Yosys` (with `ABC`), followed by gate-level simulation.
- **Part C:** Static timing analysis of the synthesised netlist using `OpenSTA`.
- **Part D:** Physical design from RTL to GDSII using `LibreLane` (`OpenROAD`, `Magic`, `KLayout`, `Netgen`).

We ran all the tools inside the IIC-OSIC-TOOLS Docker image (repository `hpretl/iic-osic-tools`, tag `latest`, image ID `306585de35d1`) on Windows 11, using Docker Desktop with WSL 2. Our design folder `~/eda/designs` in WSL shows up as `/foss/designs` inside the container. To make `sky130A` the default PDK, we added a `.designinit` file in this folder, which the container reads every time it starts. For Part D we always started LibreLane through the `librelane` wrapper that comes with the image. This wrapper picks the OpenROAD and OpenSTA builds that LibreLane is pinned to and adds `--manual-pdk`, so that the PDK already installed in the image gets used. Calling LibreLane directly with `python3 -m librelane` skips the wrapper, and when we tried that we ran into errors caused by mismatched tool versions.

Flow Stage	**Tool**	**Version**
RTL and gate-level simulation	Icarus Verilog	14.0 (devel)
Waveform viewing	GTKWave	bundled with image
Logic synthesis	Yosys + ABC	0.70
Static timing analysis (Part C)	OpenSTA	3.1.0
RTL-to-GDSII flow	LibreLane	3.1.0.dev3
Place and route engine	OpenROAD	26Q1-1024-gdcf36133a
Layout viewing and DRC	KLayout	0.30.12
LVS	Netgen	1.5.324
Process design kit	SkyWater SKY130	`sky130A`, `sky130_fd_sc_hd`

Table 1: Open-source tools and versions used in this assignment.

Our repository follows the file structure given in the assignment:

```
dvd_assignment/
|-- README.md
|-- rtl/
|   |-- full_adder.v
|   `-- mult_array.v
|-- tb/
|   `-- tb_mult.v
|-- synth/
|   |-- synth.tcl
|   |-- mult_array_netlist.v      (generated)
|   `-- mult_array_stat.rpt       (generated)
|-- sta/
|   |-- sta.tcl
|   `-- mult_array_timing.rpt     (generated)
|-- pnr/
|   |-- config.json
|   |-- metrics.json              (copied from the final run)
|   `-- mult_array.gds            (copied from the final run)
`-- report/
    `-- report.pdf
```

Listing 1: Repository File Architecture

2 Part A: RTL Design and Verification

Multiplying two 4-bit numbers is basically shift-and-add. Each bit b_i of the multiplier gives one partial product $pp_i[j] = a[j] \cdot b_i$, shifted left by i places, and adding up the four partial products gives the 8-bit product. The array multiplier does this with a row of AND gates for every multiplier bit, followed by three rows of 4-bit ripple-carry adders. In general an N-bit array needs N^2 AND gates and $N(N-1)$ full adders, so for $N = 4$ we need 16 AND gates and 12 full adders.

2.1 Task 1: Full Adder

We wrote the full adder using only `assign` statements. The sum is the XOR of all three inputs. The carry is 1 when both `a` and `b` are 1, or when exactly one of them is 1 and there is a carry coming in.

```
module full_adder (
    input  a,
    input  b,
    input  cin,
    output sum,
    output cout
);
    assign sum  = a ^ b ^ cin;
    assign cout = (a & b) | (cin & (a ^ b));
endmodule
```

Listing 2: `rtl/full_adder.v`

2.2 Task 2: Array Multiplier

The multiplier first registers `a` and `b` into two 4-bit registers on every rising edge of the clock. The array is built from these registered values, and the 8-bit result goes into the output register `p`. All 16 flip-flops are cleared by the asynchronous active-low reset `rst_n`. The three adder rows use 12 instances of `full_adder`, connected exactly as in the connection table from the assignment, and we did not use the `*` operator anywhere in this file.

```
module mult_array (
    input             clk,
    input             rst_n,
    input       [3:0] a,
    input       [3:0] b,
    output reg  [7:0] p
);
    reg  [3:0] a_reg, b_reg;              // input registers
    wire [3:0] pp0, pp1, pp2, pp3;        // partial products
    wire [3:0] r0, r1, r2;                // bits passed down between
   rows
    wire [3:0] s1, s2, s3;                // full-adder sums
    wire [4:0] c1, c2, c3;                // carries; c_i[0] tied to 0
    wire [7:0] p_comb;                    // combinational product

```

```
15      // Input registers: capture a and b on every rising edge
16      always @(posedge clk or negedge rst_n) begin
17          if (!rst_n) begin
18              a_reg <= 4'd0;
19              b_reg <= 4'd0;
20          end else begin
21              a_reg <= a;
22              b_reg <= b;
23          end
24      end
25
26      // 16 AND gates: ppi[j] = a[j] AND b[i]
27      assign pp0 = a_reg & {4{b_reg[0]}};
28      assign pp1 = a_reg & {4{b_reg[1]}};
29      assign pp2 = a_reg & {4{b_reg[2]}};
30      assign pp3 = a_reg & {4{b_reg[3]}};
31
32      // Row 0: AND gates only
33      assign p_comb[0] = pp0[0];
34      assign r0 = {1'b0, pp0[3], pp0[2], pp0[1]};
35
36      // Row 1: adds pp1 to r0
37      assign c1[0] = 1'b0;
38      full_adder fa1_0 (.a(pp1[0]), .b(r0[0]), .cin(c1[0]), .sum(s1[0]),
        .cout(c1[1]));
39      full_adder fa1_1 (.a(pp1[1]), .b(r0[1]), .cin(c1[1]), .sum(s1[1]),
        .cout(c1[2]));
40      full_adder fa1_2 (.a(pp1[2]), .b(r0[2]), .cin(c1[2]), .sum(s1[2]),
        .cout(c1[3]));
41      full_adder fa1_3 (.a(pp1[3]), .b(r0[3]), .cin(c1[3]), .sum(s1[3]),
        .cout(c1[4]));
42      assign p_comb[1] = s1[0];
43      assign r1 = {c1[4], s1[3], s1[2], s1[1]};
44
45      // Row 2: adds pp2 to r1
46      assign c2[0] = 1'b0;
47      full_adder fa2_0 (.a(pp2[0]), .b(r1[0]), .cin(c2[0]), .sum(s2[0]),
        .cout(c2[1]));
48      full_adder fa2_1 (.a(pp2[1]), .b(r1[1]), .cin(c2[1]), .sum(s2[1]),
        .cout(c2[2]));
49      full_adder fa2_2 (.a(pp2[2]), .b(r1[2]), .cin(c2[2]), .sum(s2[2]),
        .cout(c2[3]));
50      full_adder fa2_3 (.a(pp2[3]), .b(r1[3]), .cin(c2[3]), .sum(s2[3]),
        .cout(c2[4]));
51      assign p_comb[2] = s2[0];
52      assign r2 = {c2[4], s2[3], s2[2], s2[1]};
53
54      // Row 3: adds pp3 to r2 and produces the top four product bits
55      assign c3[0] = 1'b0;
56      full_adder fa3_0 (.a(pp3[0]), .b(r2[0]), .cin(c3[0]), .sum(s3[0]),
        .cout(c3[1]));
57      full_adder fa3_1 (.a(pp3[1]), .b(r2[1]), .cin(c3[1]), .sum(s3[1]),
        .cout(c3[2]));
58      full_adder fa3_2 (.a(pp3[2]), .b(r2[2]), .cin(c3[2]), .sum(s3[2]),
        .cout(c3[3]));
```

```
59      full_adder fa3_3 (.a(pp3[3]), .b(r2[3]), .cin(c3[3]), .sum(s3[3]),
        .cout(c3[4]));
60      assign p_comb[3]   = s3[0];
61      assign p_comb[7:4] = {c3[4], s3[3], s3[2], s3[1]};
62
63      // Output register: capture the 8-bit product
64      always @(posedge clk or negedge rst_n) begin
65          if (!rst_n) p <= 8'd0;
66          else        p <= p_comb;
67      end
68  endmodule
```

Listing 3: `rtl/mult_array.v`

An input that is applied before rising edge 1 gets captured into `a_reg`/`b_reg` at edge 1, and the product gets captured into `p` at edge 2. So the latency is 2 clock cycles. Since a new input can be applied in every cycle, the throughput is one result per cycle.

Block Diagram

Figure 1 shows the full structure of our design: the input registers, the 16 AND gates (one for each partial-product bit), the 12 full adders in three ripple-carry rows, and the output register. Every row sends its upper three sum bits and its final carry down to the next row, and its lowest sum bit is already a finished product bit.

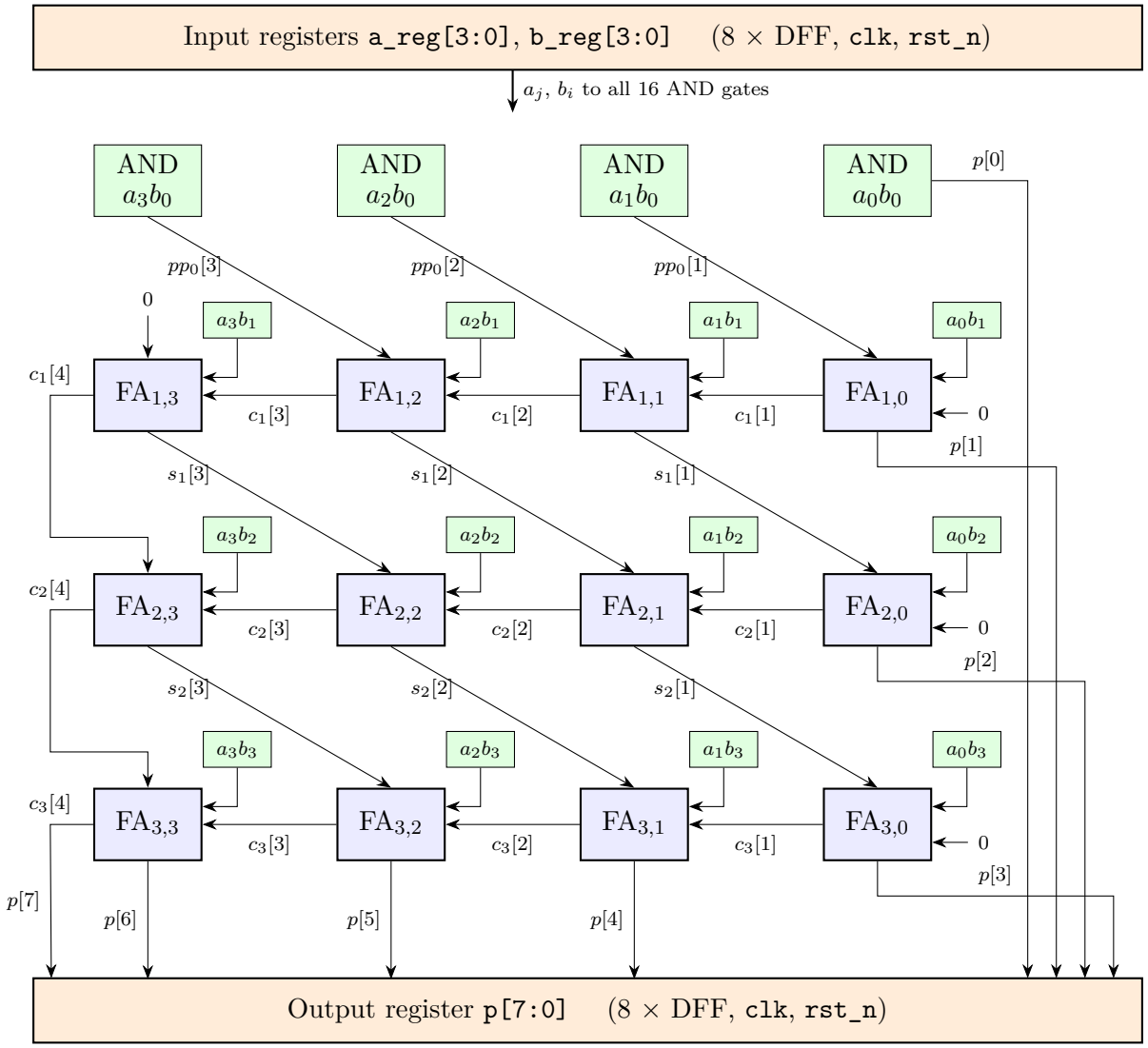


Figure 1: Block diagram of our 4-bit array multiplier, with 8 input flip-flops, 16 AND gates (green), 12 full adders in three ripple-carry rows (blue) and 8 output flip-flops. $\mathrm{FA}_{i,j}$ adds $pp_i[j]$, the bit coming down from row $i-1$, and the carry $c_i[j]$.

2.3 Task 3: Self-Checking Testbench

Our testbench `tb_mult` generates a 10 ns clock, keeps `rst_n` low for the first two cycles and then goes through all 256 input combinations. Each input pair is applied on a falling edge, so the inputs never change right at the active rising edge, and the output is checked two falling edges later, after both register stages have been clocked. We store the expected value in an 8-bit register so that the full product width gets compared (and also printed correctly in the mismatch message). This is the only place in the project where the `*` operator is used.

```
`timescale 1ns / 1ps

module tb_mult;
    reg         clk, rst_n;
    reg  [3:0] a, b;
    wire [7:0] p;
    reg  [7:0] expected;
    integer     i, j, errors, vectors;

    mult_array dut (.clk(clk), .rst_n(rst_n), .a(a), .b(b), .p(p));

    // 10 ns clock period
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        $dumpfile("mult.vcd");
        $dumpvars(0, tb_mult);
        errors = 0; vectors = 0; a = 0; b = 0;

        // Hold reset low for the first 2 clock cycles
        rst_n = 0;
        #20 rst_n = 1;

        // Exhaustive test: all 256 input combinations
        for (i = 0; i < 16; i = i + 1) begin
            for (j = 0; j < 16; j = j + 1) begin
                @(negedge clk);                 // apply inputs away
   from the rising edge
                a = i[3:0];
                b = j[3:0];
                expected = i * j;               // * is allowed in the
   testbench only
                @(negedge clk);                 // rising edge 1:
   inputs registered
                @(negedge clk);                 // rising edge 2:
   product registered
                if (p !== expected) begin
                    $display("Mismatch: a=%0d b=%0d expected=%0d got=%0
   d", a, b, expected, p);
                    errors = errors + 1;
                end
```

```
                vectors = vectors + 1;
            end
        end

        if (errors == 0)
            $display("PASS: %0d vectors checked, 0 errors", vectors);
        else
            $display("FAIL: %0d vectors checked, %0d errors", vectors,
    errors);
        $finish;
    end
endmodule
```

Listing 4: `tb/tb_mult.v`

Simulation Results

We compiled and ran the simulation with the commands below, and every vector passed:

```
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim
# PASS: 256 vectors checked, 0 errors
```

Listing 5: RTL Simulation Commands and Result

2.4 Task 4: Error-Detection Test

To make sure the testbench can actually catch bugs, we temporarily changed the sum in `full_adder.v` from `a ^ b ^ cin` to `a ^ b`, so the carry-in is ignored in every sum. The testbench then printed a list of mismatches, starting with `a=3 b=3 expected=9 got=5`, and ended with:

```
FAIL: 256 vectors checked, 95 errors
```

Listing 6: Testbench Output with the Deliberate Bug

The other 161 vectors still passed. These are the cases where no full adder ever gets a carry-in of 1. For example, if either operand is 0 or a power of two, only one partial product is non-zero, so no carries are produced and the bug does not show up. After the test we undid the change with `git checkout -- rtl/full_adder.v`, and the testbench printed `PASS` again.

2.5 Task 5: Worked Example and Waveform

For the example we used the roll number **2024112012** (Vishnubhotla Nitin Sri Siva Kalyan), so $A = 2 + 5 = 7$. Since A came out as 7, the rule says to use $B = 9$. Working it out by hand in the same layout as the assignment:

				0	1	1	1	$A = 7$
			$\times$	1	0	0	1	$B = 9$
				0	1	1	1	$pp_0 = A \cdot b_0$ $(b_0 = 1)$
			0	0	0	0		$pp_1 = A \cdot b_1$ $(b_1 = 0)$, shifted 1
		0	0	0	0			$pp_2 = A \cdot b_2$ $(b_2 = 0)$, shifted 2
	0	1	1	1				$pp_3 = A \cdot b_3$ $(b_3 = 1)$, shifted 3
0	0	1	1	1	1	1	1	$P = 7 + 56 = \mathbf{63}$

Table 2: Hand-worked multiplication $7 \times 9 = 63$.

Only pp_0 and pp_3 are non-zero here, so the product is $7 + 7 \cdot 2^3 = 63 = 0011\,1111_2$. In the exhaustive sweep this is vector number 121 ($a = 7$, $b = 9$). Figure 2 shows it in GTKWave. `b` changes from 8 to 9 at 3650 ns. Marker A (3655 ns) is rising edge 1, where the inputs get registered, and marker B (3665 ns) is rising edge 2, where `p` changes from 56 (the previous product, 7×8) to **63**. So the product shows up two clock cycles after the inputs are applied, which matches the 2-cycle latency we expected. `rst_n` stays high during this whole window.

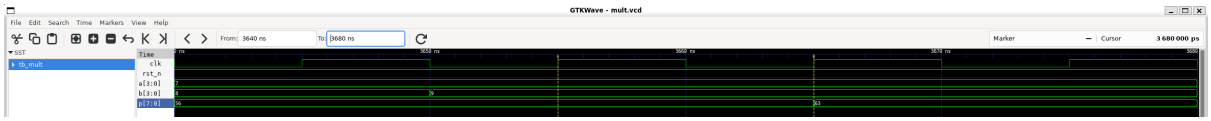


Figure 2: GTKWave view of `clk`, `rst_n`, `a`, `b` and `p` for 7×9. Marker A (3655 ns) is rising edge 1, where the inputs are registered. Marker B (3665 ns) is rising edge 2, where `p` becomes 63. Latency = 2 clock cycles.

3 Part B: Logic Synthesis with Yosys

Yosys converts our RTL into a netlist of SKY130 standard cells. Our script follows the seven steps listed in the assignment and uses the typical-corner Liberty file `sky130_fd_sc_hd__tt_025C_1v80.lib`. It does generic synthesis (flattening the full adders into one module), maps the flip-flops with `dfflibmap`, maps the rest of the logic with `abc`, removes unused wires, and then writes out the area report and the gate-level netlist.

3.1 Task 1: Synthesis Script

```
# Part B: synthesise mult_array to SKY130 standard cells (typical
    corner)
set lib $::env(PDK_ROOT)/sky130A/libs.ref/sky130_fd_sc_hd/lib/
    sky130_fd_sc_hd__tt_025C_1v80.lib

yosys read_verilog rtl/full_adder.v rtl/mult_array.v
yosys synth -top mult_array -flatten
yosys dfflibmap -liberty $lib
yosys abc -liberty $lib
yosys opt_clean -purge
yosys tee -o synth/mult_array_stat.rpt stat -liberty $lib
yosys write_verilog -noattr synth/mult_array_netlist.v
```

Listing 7: `synth/synth.tcl`

We ran the script with `yosys -c synth/synth.tcl`. Because of the flattening, the netlist has only one module (`mult_array`), and the 12 `full_adder` instances are no longer there as separate blocks.

3.2 Task 2: Synthesis Results

Metric	Value
Total number of cells	67
Number of flip-flops	16 (`sky130_fd_sc_hd__dfrtp_1`)
Total cell area	$784.502\,\mu\text{m}^2$
Flip-flop area	$16 \times 25.024 = 400.384\,\mu\text{m}^2$
Flip-flop area (% of total)	**51.04%**
The 3 most-used cell types	`dfrtp_1` (16), `nand2_1` (9), `xnor2_1` (7)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

Table 3: Part B results from `synth/mult_array_stat.rpt`.

The remaining 51 combinational cells ($384.118\,\mu\text{m}^2$ in total) are made up of these cells:

Cell	Count	Cell	Count	Cell	Count
nand2_1	9	a22oi_1	3	a2111oi_0	1
xnor2_1	7	a21oi_1	3	nand3_1	1
maj3_1	5	and4_1	2	nand4_1	1
and2_0	4	clkinv_1	2	nor2b_1	1
xnor3_1	2	lpflow_isobufsrc_1	2	or3_1	1
mux2i_1	2	nor2_1	2	xor3_1	1
o21ai_0	2				

Table 4: Combinational cells in the synthesised netlist (all `sky130_fd_sc_hd__` prefixed).

3.3 Task 3: Gate-Level Simulation

To check that synthesis did not change the behaviour of the design, we ran the same testbench on the synthesised netlist, using the SKY130 functional cell models instead of our RTL:

```
CELLS=$PDK_ROOT/sky130A/libs.ref/sky130_fd_sc_hd/verilog
iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=#1 -o gls \
    tb/tb_mult.v $CELLS/primitives.v $CELLS/sky130_fd_sc_hd.v synth/
   mult_array_netlist.v
vvp gls
# PASS: 256 vectors checked, 0 errors
```

Listing 8: Gate-Level Simulation Commands and Result

4 Part C: Static Timing Analysis with OpenSTA

OpenSTA checks every timing path in the synthesised netlist against the clock period, without needing any simulation. We only used the typical corner (`tt_025C_1v80`). All inputs except the clock are assumed to arrive 1.0 ns after the clock edge, and the outputs have to be ready 1.0 ns before the next edge.

4.1 Task 1: STA Script and Maximum Frequency

```
# Part C: static timing analysis of the synthesised netlist (typical
    corner)
set lib $::env(PDK_ROOT)/sky130A/libs.ref/sky130_fd_sc_hd/lib/
    sky130_fd_sc_hd__tt_025C_1v80.lib

read_liberty $lib
read_verilog synth/mult_array_netlist.v
link_design mult_array

create_clock -name clk -period 2.98 [get_ports clk]
set_input_delay 1.0 -clock clk [delete_from_list [all_inputs] [
    get_ports clk]]
set_output_delay 1.0 -clock clk [all_outputs]

report_worst_slack -max
report_checks -path_delay max -digits 3 > sta/mult_array_timing.rpt
exit
```

Listing 9: `sta/sta.tcl` (final version at the minimum period of 2.98 ns)

We first ran the script with `sta sta/sta.tcl` at $T = 10$ ns, and it reported a worst setup slack of WNS $= 7.02$ ns. From this:

$$T_{min} = T - \text{WNS} = 10 - 7.02 = 2.98 \text{ ns}, \qquad F_{max} = \frac{1000}{T_{min}} = \frac{1000}{2.98} = 335.57 \text{ MHz}$$

When we ran it again with the period set to 2.98 ns, the worst slack came out as **0.000 ns (MET)**, which confirms T_{min}. We did not have to round anything here, because the critical path needs exactly 2.844 ns of arrival time plus 0.136 ns of setup time, which adds up to 2.980 ns.

The timing report has two path groups. The `asynchronous` group is the recovery check from the `rst_n` input to the reset pin of a flip-flop (slack 2.208 ns at T_{min}). This is not a data path, so it does not decide the clock period. The `clk` group has the register-to-register critical path, and this is the one that sets T_{min}.

4.2 Task 2: Critical Path

The critical path starts at flip-flop `_106_`, whose output is `b_reg[0]` (the multiplier bit b_0), and ends at flip-flop `_100_`, whose output is `p[6]`. The D input of `_100_` is `fa3_3.sum`, which is $s_3[3]$ in our RTL. So the path starts at b_0 and goes through all three adder rows to the last full adder in row 3.

#	Instance	Cell	Delay (ns)	Arrival (ns)
–	`_106_` (launch, `b_reg[0]`)	`dfrtp_1` clock-to-Q	0.411	0.411
1	`_048_`	`and4_1`	0.218	0.629
2	`_050_`	`a2111oi_0`	0.428	1.057
3	`_051_`	`a21oi_1`	0.094	1.151
4	`_053_`	`mux2i_1`	0.218	1.369
5	`_069_`	`xnor3_1`	0.374	1.742
6	`_074_`	`maj3_1`	0.371	2.113
7	`_076_`	`maj3_1`	0.376	2.489
8	`_080_`	`xor3_1`	0.355	2.844
–	`_100_` (capture, `p[6]`)	`dfrtp_1` setup time	0.136	2.980

Table 5: Cells on the critical path from `sta/mult_array_timing.rpt` ($T = 2.98$ ns, slack 0.000 ns).

Out of the 2.98 ns period, 0.411 ns is the clock-to-Q delay of the launching flip-flop, **2.433 ns** is spent in the 8 logic cells, and 0.136 ns is the setup time of the capturing flip-flop.

4.3 Task 3: Comparison with the $2N$ Full-Adder Estimate

Only **4** of the 8 logic cells on the path actually implement full-adder functions: `xnor3_1` and `xor3_1` compute sums of three inputs, and the two `maj3_1` cells compute carries (the majority of three inputs). The other 4 cells (`and4_1`, `a2111oi_0`, `a21oi_1`, `mux2i_1`) are where the partial-product AND gates got merged with the first adder stages.

This means that the estimate of about $2N = 8$ full-adder delays is more of an upper bound for our design. In the RTL, each full adder would need at least two levels of gates (an XOR for the sum and some AND-OR logic for the carry). After flattening, however, ABC was able to implement a whole sum or carry with a single 3-input cell, and it also merged the simpler adders of the first row (which have a carry-in of 0, and `fa1_3` also has $r_0[3] = 0$) into compound gates. So the real path is more like 4 full-adder cells plus 4 cells of merged partial-product logic, with an average delay of about 0.304 ns per cell.

4.4 Part C Results

Metric	Value
Worst slack at $T = 10\,\text{ns}$	7.02 ns
T_{min}	2.98 ns
F_{max}	**335.57 MHz**
Slack at $T = T_{min}$	0.000 ns
Critical path start point	`b_reg[0]` (`_106_`, `dfrtp_1`)
Critical path end point	`p[6]` (`_100_`, `dfrtp_1`)
Number of cells on the critical path	8 logic cells (plus launch and capture flip-flops)
Latency $= 2 \times T_{min}$	5.96 ns
Throughput	335.57 million results per second

Table 6: Part C results at the typical corner.

5 Part D: Physical Design from RTL to GDSII

LibreLane runs the whole physical design flow for us: synthesis, floorplanning (including the power grid), placement, clock tree synthesis, routing, parasitic extraction and the signoff checks (STA, DRC and LVS), and at the end it writes out the GDSII layout. Internally it uses Yosys, OpenROAD, Magic, KLayout and Netgen.

5.1 Task 1: Configuration and Runs

We set the clock period to $1.2 \times T_{min} = 1.2 \times 2.98 = 3.576\,\text{ns}$, rounded up to the next 0.5 ns, which gives **4.0 ns**. The die is fixed at $150\,\mu\text{m} \times 150\,\mu\text{m}$ because the design is very small.

```
{
    "DESIGN_NAME": "mult_array",
    "VERILOG_FILES": ["dir::../rtl/full_adder.v", "dir::../rtl/
   mult_array.v"],
    "CLOCK_PORT": "clk",
    "CLOCK_PERIOD": 4.0,
    "FP_SIZING": "absolute",
    "DIE_AREA": [0, 0, 150, 150],
    "TIMING_VIOLATION_CORNERS": ["*tt*"]
}
```

Listing 10: `pnr/config.json` (final version)

We ran the flow from the `pnr/` folder with `librelane config.json`. It took two runs, both with `CLOCK_PERIOD` = 4.0 ns:

Run	Configuration	Setup slack (ns)			Result
		tt	ff	ss	
1	Required settings only	+0.883	+1.975	−1.926	Stopped at the ss-corner setup check
2	Added `TIMING_VIOLATION_CORNERS = ["*tt*"]`	+0.883	+1.975	−1.926	**Flow complete**

Table 7: LibreLane runs (nominal corners: `tt_025C_1v80`, `ff_n40C_1v95`, `ss_100C_1v60`).

In run 1, all 80 stages finished with 0 DRC errors, 0 LVS errors and a passing antenna check. However, the flow stopped at the end because of setup violations at the three slow corners (`ss_100C_1v60`, i.e. 100 °C and 1.60 V). With this PDK setup, LibreLane checks setup timing at every corner by default, while in Part C we only looked at the typical corner.

For run 2 we added `"TIMING_VIOLATION_CORNERS": ["*tt*"]`, so that the signoff timing check uses the same corner as Part C. LibreLane still runs STA at every corner and still reports the slack for all of them; only the pass/fail decision changes. The layout is the same in both runs. At the end we copied `metrics.json` and `mult_array.gds` from `runs/<run>/final/` into `pnr/`.

5.2 Task 2: Physical Design Results

Metric	Value
`CLOCK_PERIOD` used	4.0 ns
Number of runs needed	2
Core area	17 759.5 μm^2 (die: 22 500 μm^2)
Cells after synthesis (Part B)	67
Cells after place and route	1898
Kinds of cells added	decap, filler, well tap, buffers, clock buffers
Wirelength (detailed route)	2378 μm
Setup slack (`nom_tt_025C_1v80`)	+0.883 ns
Post-route F_{max}	**320.8 MHz**
F_{max} from Part C	335.57 MHz
DRC errors	0 (routing, Magic and KLayout)
LVS errors	0 (circuits match uniquely)

Table 8: Part D results from `pnr/metrics.json`.

5.3 Task 3: Post-Route Maximum Frequency

$$F_{max,\,post\text{-}route} = \frac{1000}{\texttt{CLOCK_PERIOD} - \text{slack}} = \frac{1000}{4.0 - 0.883} = \frac{1000}{3.117} = 320.8 \text{ MHz}$$

The post-route F_{max} is **0.96**$\times$ the Part C value, i.e. 4.4% lower, since the effective minimum period goes up from 2.98 ns to 3.117 ns. Both numbers are for the typical corner, so the comparison is fair. We go through the reasons for this in Question 4.

5.4 Task 4: Cells Added by Place and Route

LibreLane runs its own synthesis step, which gave 68 cells (4 inverters, 16 flip-flops and 48 multi-input combinational cells) instead of the 67 we got in Part B. The other $1898 - 68 = 1830$ cells were added during physical design:

Kind	**Cells in the final layout**	**Count**
Decoupling capacitors	`decap_12` (891), `decap_6` (181), `decap_3` (129), `decap_8` (51), `decap_4` (41)	1293
Filler	`fill_1` (216), `fill_2` (41)	257
Well taps	`tapvpwrvgnd_1`	245
Timing-repair buffers	`buf_2` (17), `buf_4` (12), `buf_6` (1), `clkdlybuf4s50_1` (2)	32
Clock-tree buffers	`clkbuf_16`	3
Total added		**1830**

Table 9: Cells added during place and route (from the final DEF and the `instance__count__class` metrics).

Apart from adding cells, the resizer also swapped some existing cells for bigger (stronger) versions without changing the cell count. For example, three of the flip-flops became `dfrtp_2`, and some of the majority gates became `maj3_2` or `maj3_4`.

5.5 Task 5: Final Layout

Figure 3 shows the whole die and Figure 4 shows a zoomed-in part of the core. Both screenshots were taken in KLayout with the SKY130 layer properties loaded.

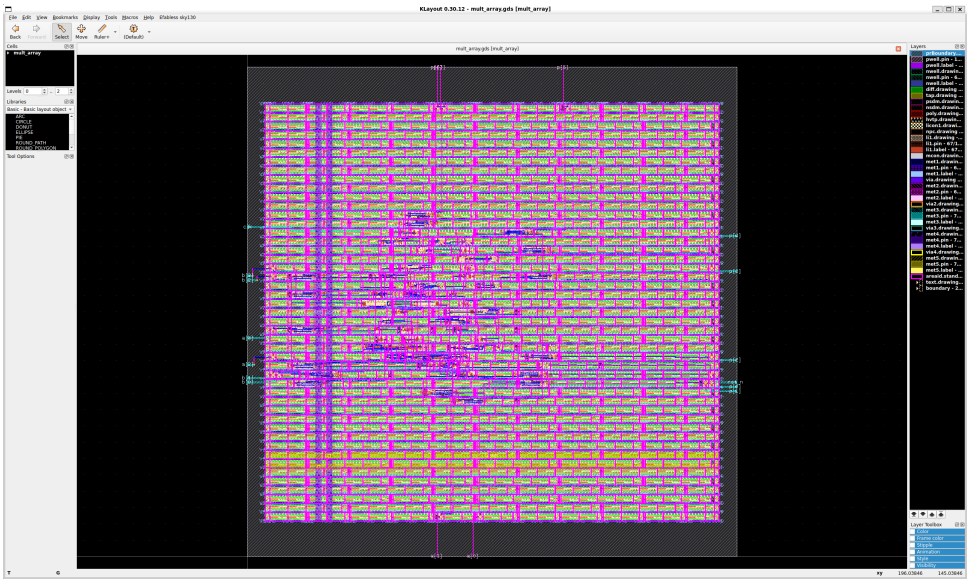


Figure 3: Final layout of the whole die. The standard-cell core sits inside the 150 μm $\times$ 150 μm die, the I/O pins are on the die edges (`clk`, `a` and `b` on the left, `p` and `rst_n` on the right, top and bottom), and the vertical stripes are the power straps.

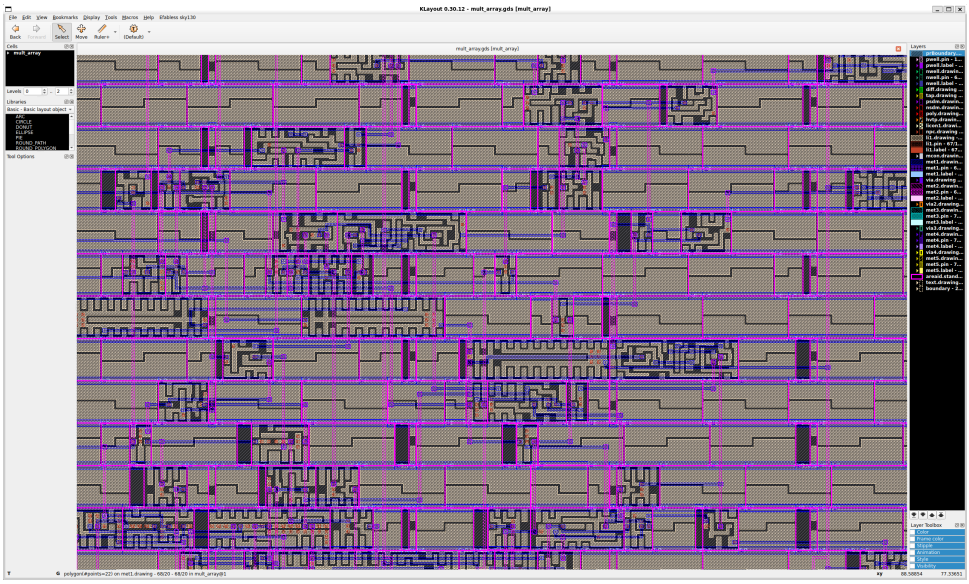


Figure 4: Zoomed view of the core. The standard-cell rows are bounded by `met1` power rails. The logic cells (with dense internal geometry) sit between decap and filler cells (plain hatched blocks), and they are connected by horizontal `met1` and vertical `met2` wires.